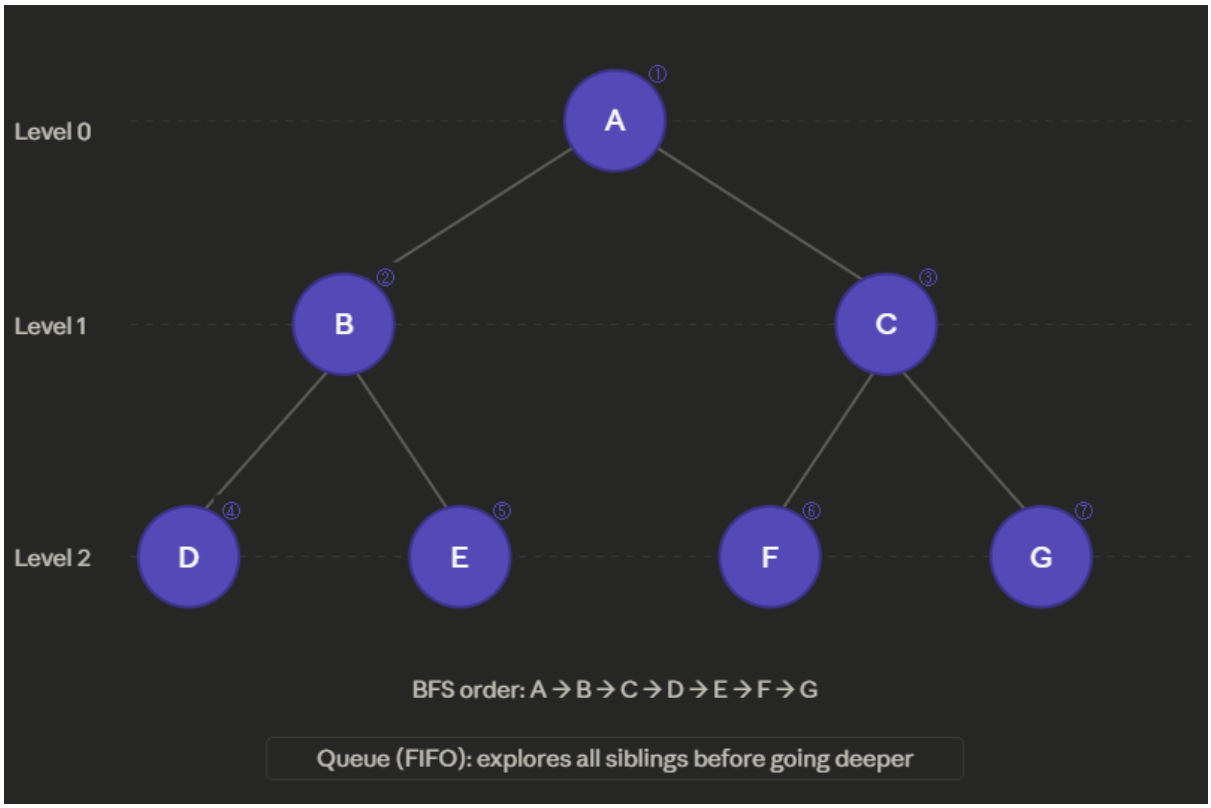
breadth-first search algorithm

BFS is a graph/tree traversal algorithm that explores all neighbors at the current depth before moving to nodes at the next depth level. It uses a **queue** (FIFO) to track nodes to visit.



Pseudocode

```
BFS(graph, start):
  queue ← [start]
  visited ← {start}
while queue not empty:
    node ← queue.dequeue()
    process(node)
 for each neighbor of node:
      if neighbor not in visited:
        visited.add(neighbor)
        queue.enqueue(neighbor)
```

Complexity & Trade-offs

Let **V** = number of vertices, **E** = number of edges.

- Time complexity

  O(V + E)

  Every vertex and edge visited once

- Space complexity

  O(V)

  Queue + visited set, worst case all V nodes

Advantages

- Guarantees shortest path in unweighted graphs
- Will never get stuck in an infinite loop (with visited set)
- Finds solution closest to root first
- Good for dense, shallow graphs

Disadvantages

- High memory usage — holds entire frontier in queue
- Slower than DFS when target is deep in the graph
- Does not work well with weighted edges (use Dijkstra instead)
- Less cache-friendly than DFS due to wide traversal